



UNIVERSITÀ DEGLI STUDI
DI SALERNO

Corso di Laurea in Ingegneria Informatica

DOCUMENTO DI PROGETTAZIONE

Applicazione Mobile Cross-Platform “MealCraft”

Corso di **Mobile Programming** – III Anno Accademico

Docente del Corso:
Prof. Francesco Cauteruccio

Sviluppato dal Gruppo N. 8:

Roberti Roberto
Mat. 0612709470

Iandoli Felice
Mat. 0612709720

Ricciardelli Jacopo Pio
Mat. 0612709557

De Santis Andrea
Mat. 0612709693

Anno Accademico 2025 – 2026

Indice

1	Descrizione del Gruppo	2
2	Descrizione dell'Obiettivo e della Struttura	2
3	Requisiti	2
4	Progettazione dell'App	3
4.1	Schermata Dashboard Analitica (index)	4
4.2	Schermata Gestione Dispensa (pantry)	4
4.3	Schermata Calendario e Pianificazione (planner)	5
4.4	Schermata Ricettario Digitale (recipes)	6
4.5	Schermata Lista della Spesa (shopping)	6
5	Flussi di Navigazione e Architettura dei Canali	7
5.1	Il Flusso di Pianificazione Alimentare (Planner → Picker → Planner)	7
5.2	Il Flusso di Approvvigionamento e Carico (Shopping → Dispensa)	8
5.3	Il Flusso di Monitoraggio Centrale (Home → Navigazione Radiale)	8
6	Organizzazione dei Dati e Strutture Core	9
7	Wireframe e Layout dell'Applicazione	10
8	Wireflow e Diagrammi di Flusso	12
9	Framework Utilizzato e Stack Tecnologico	14
9.1	Librerie Principali e Dipendenze	14
9.2	Motivazione delle Scelte Architettureali	15
9.3	Gestione della Persistenza dei Dati	15
10	Implementazione e Struttura del Software	16
10.1	Organizzazione del Codice (Separation of Concerns)	16
10.2	Componenti e Widget Grafici Principali	17
10.3	Delucidazioni Tecniche su Blocchi di Codice Complessi	17
11	Considerazioni Finali	21
11.1	Possibili implementazioni future	21

1. Descrizione del Gruppo

INFO GENERALI

- DOCUMENTO DI PROGETTAZIONE DEL CORSO DI “MOBILE PROGRAMMING” DEL III ANNO ACCADEMICO.
- MealCraft è un progetto incentrato sulla realizzazione di un app che permette la navigazione agli utenti con l’obiettivo di gestire la propria dispensa, la pianificazione dei pasti, le proprie abitudini alimentari e stilare la lista della spesa.
- Il lavoro è svolto dal gruppo numero 8 composto dai seguenti componenti:
 - Roberti Roberto - Matricola: 0612709470
 - Iandoli Felice - Matricola: 0612709720
 - Ricciardelli Jacopo Pio - Matricola: 0612709557
 - De Santis Andrea - Matricola: 0612709693

2. Descrizione dell’Obiettivo e della Struttura

STRUTTURA GENERALE

L’applicazione ha come obiettivo quello di aiutare gli utenti alla pianificazione della propria settimana alimentare: infatti coloro che utilizzano l’app devono poter organizzare i pasti, gestire le ricette e monitorare ingredienti e prodotti disponibili in dispensa. Inoltre è inserita l’opzione di pianificazione dei pasti, la creazione di una lista della spesa e la presenza di elementi riepilogativi sulle proprie abitudini alimentari. È rivolta a tutti gli utenti e, inoltre si può utilizzare in vari scenari:

- Una persona che segue un regime alimentare ben definito può pianificare i pasti giornalieri e consultare la dispensa per capire se deve recarsi a fare la spesa per mancanza di prodotti;
- Una famiglia ha la possibilità di sfruttare l’app per avere più organizzazione generale sugli acquisti da fare e i pasti da preparare;
- È possibile inoltre consultare il ricettario per avere un accesso rapido a procedimenti che non si ricordano o ad ingredienti che mancano.

3. Requisiti

FUNZIONALITÀ IMPLEMENTATE

Nello sviluppo dell’applicazione abbiamo considerato tante funzionalità da implementare ma alla fine la scelta è ricaduta sulle seguenti:

- Inserimento, Modifica o Eliminazione di ricette;
- Visualizzazione dell’elenco ricette;
- Consultazione del dettaglio della ricetta;
- Inserimento, Modifica o Eliminazione di prodotti in dispensa;

- Visualizzazione dei prodotti presenti, prodotti prossimi alla scadenza e prodotti scaduti;
- Aggiornamento di quantità e grammature dei prodotti;
- Inserimento di pasti pianificati;
- Aggiornamento del piano di pianificazione con eliminazione di un pasto;
- Consultazione delle ricette associate ai pasti;
- Visualizzazione della lista della spesa;
- Inserimento, Modifica o Eliminazione di elementi;
- Spuntare l'acquisto degli elementi;
- Ricerca di ricette filtrata per nome, categoria e tempo di preparazione;
- Ricerca di ingredienti disponibili e prodotti prossimi alla scadenza;
- Ricerca di pasti pianificati per un determinato giorno;
- Ricerca di elementi della lista della spesa;
- Grafici riepilogativi delle statistiche e delle abitudini alimentari;
- Elementi riassuntivi dell'organizzazione logica dietro alla pianificazione dei pasti.

FUNZIONALITÀ NON IMPLEMENTATE / LIMITAZIONI NOTE

In fase di confronto iniziale è stata valutata l'integrazione di un sistema di scanning di codici QR, creati appositamente per indicare i prodotti, tramite fotocamera per l'inserimento rapido nella lista della spesa; inoltre è stata considerata anche l'idea della creazione di un database in SQL. Tali funzionalità sono state temporaneamente escluse per un ambiente maggiormente circoscritto ma ciò non esclude possibili implementazioni e sviluppi software in futuro.

4. Progettazione dell'App

SCHERMATE GENERALI

L'app MealCraft è formata da 5 schermate principali che implementano la navigazione tra di loro in modo tale da permettere all'utente di effettuare interazioni tra pagine differenti e navigare tra loro. Le 5 schermate sono le seguenti:

1. Index/Home
2. Pantry (Dispensa)
3. Planner (Pianificazione)
4. Recipes (Ricette)
5. Shopping (Lista della Spesa)

4.1 Schermata Dashboard Analitica (index)

La **HomeScreen** (denominata commercialmente "*Cucina Smart*") funge da cruscotto analitico e centro di controllo dell'intera applicazione. Essa non è predisposta per l'immissione diretta dei dati, bensì opera come un modulo di visualizzazione intelligente che aggrega e computa le informazioni grezze del **Context** (Pasti, Dispensa, Ricette), traducendole in metriche grafiche.

Le sue funzionalità core includono:

- **Griglia delle Statistiche Rapide:** Mostra tre contatori sintetici immediati relativi al numero totale di ricette archiviate, ai pasti pianificati nella settimana corrente e al tempo medio di preparazione stimato. Ogni scheda implementa un link di navigazione diretta alla rispettiva sezione funzionale.
- **Selettore Temporale (Toggle Week/Month):** Consente di modificare dinamicamente l'orizzonte temporale dei moduli grafici, permettendo lo switch reattivo tra la settimana in corso e l'intero mese solare.
- **Agenda del Giorno:** Visualizza in ordine cronologico i pasti pianificati per la data odierna, categorizzandoli per tipologia (**COLAZIONE**, **PRANZO**, **CENA**, **SPUNTINI**) e calcolando in tempo reale il tempo di attività e preparazione richiesto.
- **Istogramma dei Prodotti più Pianificati:** Grafico a barre generato nativamente tramite fogli di stile (*StyleSheet*), che mappa i quattro prodotti alimentari strutturalmente più ricorrenti nella pianificazione dell'utente.
- **Grafico a Torta dei Consumi:** Diagramma circolare deputato alla visualizzazione delle percentuali di consumo dei singoli ingredienti. L'algoritmo ripartisce i gradi di rotazione della componente grafica in funzione delle quote vettoriali derivate dallo stato.
- **Sezione Scadenze e Allarmi:** Modulo di monitoraggio predittivo diviso in due sottomoduli:
 - *Mancanti Settimanali:* Esegue un *diff* algoritmico tra gli ingredienti necessari per i pasti pianificati nei successivi 7 giorni e le giacenze reali in dispensa, evidenziando le carenze stock.
 - *Prodotti Critici:* Carosello orizzontale reattivo che mostra i cibi già scaduti (marcati da un *badge* rosso) o in fase di scadenza imminente entro le 72 ore successive (*badge* arancione).

4.2 Schermata Gestione Dispensa (pantry)

La **PantryScreen** sovrintende all'inventario e alla tracciabilità dei beni alimentari in possesso dell'utente. Il modulo supporta le operazioni standard di **CRUD** (Create, Read, Update, Delete), un sistema multi-livello di filtraggio in tempo reale e una macchina a stati interna per variare il comportamento in base al contesto di attivazione.

La schermata prevede due modalità operative macro:

1. **Modalità Standard (Consultazione):** Mostra il titolo "*Dispensa Intelligente*" e sblocca i permessi amministrativi completi di inserimento, modifica strutturale e rimozione fisica dei lotti alimentari.

2. **Modalità Picker (*Selection Mode*):** Si attiva qualora l'utente acceda alla dispensa a partire dal modulo di pianificazione (stato intercettato tramite l'hook `useLocalSearchParams`). Il titolo si adatta dinamicamente per riflettere il pasto bersaglio (es. *"Scegli per PRANZO"*) e inibisce i tasti di modifica/eliminazione per fare spazio a un'azione di *"SELEZIONA"*, la quale serializza l'ingrediente e lo rimanda al calendario.

Il motore di ricerca e filtraggio agisce in tempo reale tramite memorizzazione algoritmica (`useMemo`) su tre livelli combinabili:

- **Ricerca Testuale:** Filtra l'array dei prodotti basandosi sulla corrispondenza della stringa inserita nella barra di ricerca.
- **Filtro Categorie Orizzontale:** Isola i prodotti afferenti a specifiche macro-aree merceologiche (*Ortofrutta; Latticini, Salumi e Formaggi; Carne e Pesce; Secco/Dispensa; Surgelati; Bevande; Altro*) mediante filtri visivi dotati di icone dedicate.
- **Filtro di Stato (*Status Tab*):** Segmenta i prodotti in base alle condizioni temporali e quantitative:
 - *Tutti:* Visualizzazione non filtrata del vettore dispensa.
 - *In Buono Stato (Verde):* Prodotti con ciclo di vita residuo superiore ai 3 giorni.
 - *Scadenze (Arancione):* Prodotti con scadenza critica imminente (≤ 3 giorni).
 - *Stock Basso (Rosso):* Articoli con quantità inferiori o uguali a 2 unità (rischio imminente di esaurimento scorte), lotti già scaduti o disponibilità quantitativa limitata (500ml/500g).

L'interfaccia si completa di una `FlatList` di schede (`itemCard`) dotate di *border-styling* semantico (Verde, Arancione, Rosso) per riflettere lo stato del prodotto, di un form modale assistito da `KeyboardAvoidingView` per l'aggiunta/modifica dei dati e di un sistema di *Custom Alert* a tre scenari (*Successo, Errore, Avviso*) per validare la correttezza logica delle date inserite rispetto al tempo presente.

4.3 Schermata Calendario e Pianificazione (planner)

La `PlannerScreen` costituisce il modulo di pianificazione e gestione della dieta cronoprogrammata. Consente di mappare e strutturare la routine alimentare associando ricette complesse o singoli ingredienti di dispensa ai quattro slot temporali prefissati.

L'interazione si articola su quattro componenti logiche:

- **Navigazione Temporale:** Sfrutta un selettore settimanale orizzontale che calcola dinamicamente i 7 giorni della settimana corrente centrati sulla data attiva (`generateWeekDays`), affiancato da un calendario interattivo a schermo intero (`react-native-calendars`) integrato in un modulo a comparsa per salti temporali arbitrari.
- **Architettura degli Slot:** La visualizzazione giornaliera è ripartita nei blocchi COLAZIONE, PRANZO, CENA e SPUNTINI. Qualora uno slot risulti vuoto, viene renderizzato un segnaposto dichiarativo (*"Nessun elemento aggiunto"*). Ogni record presente può essere rimosso tramite trigger della funzione globale `removeFromPlan`.

- **Integrazione dei Flussi Esterni:** La pressione del pulsante di aggiunta invoca un `Bottom Sheet Modal` che espone due canali di reindirizzamento: *"Usa una Ricetta"* (verso `/recipes`) e *"Dalla Dispensa"* (verso `/pantry`), iniettando nell'URL il parametro `mealType`.
- **Parsing Reattivo al Focus:** Sfruttando gli hook combinati `useFocusEffect` e `useCallback`, la schermata intercetta il focus di ritorno dal *picker*. Se l'URL contiene dati validi, esegue il parsing dell'oggetto, genera una chiave univoca (`instanceId` basato su *timestamp*) per prevenire collisioni e invoca la funzione `updatePlan`, provvedendo immediatamente dopo alla pulizia dei parametri di navigazione per prevenire loop di inserimento accidentali.

4.4 Schermata Ricettario Digitale (recipes)

La `RecipesScreen` implementa l'archivio culinario dell'utente. Strutturata con un design ad espansione dinamica, supporta la duplice modalità operativa (consultazione/selezione) per interfacciarsi nativamente con il calendario dei pasti.

I suoi pilastri funzionali comprendono:

- **Gestione del Contesto:** In modalità standard abilita la creazione e modifica delle ricette; in modalità *picker* inibisce le funzioni di scrittura e introduce un bottone contestuale di selezione ancorato al pasto selezionato.
- **Visualizzazione Elastica:** Basata su una `FlatList` di elementi compatti. Al `onPress` di una scheda, il sistema muta lo stato `selectedId` attivando un'animazione di espansione che rivela la descrizione estesa, la lista degli ingredienti comprensiva di dosi, il procedimento passo-passo e le note di esecuzione.
- **Form di Compilazione Dinamica:** Modale a schermo intero dotato di campi testuali, input numerici per porzioni e tempistiche, e un sotto-sistema reattivo per la gestione degli ingredienti: la pressione del tasto *" + AGGIUNGI INGREDIENTE "* espande dinamicamente lo stato locale iniettando coppie di campi di testo (*Nome* e *Quantità*) all'interno del form.
- **Safe Delete Modal:** Flusso di interazione protetto da una modale di conferma esplicita per impedire la cancellazione distruttiva e irreversibile del record in caso di tocchi involontari.

Al fine di garantire un'esperienza utente immediata fin dal primo avvio, l'applicazione integra un ricettario nativo di base. Questo modulo preconfigurato è implementato tramite il file `recipes.ts` posizionato all'interno della directory `constants`. Il file contiene un dataset statico di 75 ricette selezionate tra le più comuni e diffuse, equamente ripartite in ragione di 15 ricette per ciascuna delle macro-categorie gastronomiche previste dal sistema. Tale accorgimento consente all'utente di disporre istantaneamente di un archivio culinario di partenza per la pianificazione dei pasti, senza l'obbligo di inserimento manuale iniziale.

4.5 Schermata Lista della Spesa (shopping)

La `ShoppingScreen` gestisce il ciclo di approvvigionamento delle materie prime. Oltre alla funzione di annotazione, implementa un canale di comunicazione bidirezionale verso la dispensa per automatizzare il carico di magazzino al momento dell'acquisto reale.

La schermata si articola sulle seguenti specifiche:

- **Pannello di Input Intelligente:** Permette l’inserimento di nome, quantità e unità di misura. Il software implementa un controllo logico cablato sul tipo di unità: selezionando il formato *”pezzo (pz)”*, il campo relativo al peso/volume viene disabilitato e opacizzato via codice per prevenire input ridondanti o logicamente inconsistenti.
- **Struttura Dati SectionList:** I prodotti vengono visualizzati e raggruppati all’interno di una `SectionList` ordinata per categorie merceologiche, ognuna caratterizzata da un’intestazione con codice colore semantico. La riga supporta controlli rapidi incrementali (+/−) e intercettazione di vettori esterni (parametro `params.addItem`) convogliati automaticamente nella sezione *”Altro”*.
- **Modale di Spunta e Computo Scadenze:** All’atto dell’acquisto, l’utente interagisce con una modale di rettifica che permette di aggiornare i pesi reali e determinare la data di scadenza tramite due modalità operative gestite da un `Toggle Tab`:
 - *Scadenza Relativa (+ Giorni):* Calcola la data finale eseguendo una somma algebrica tra i giorni di validità digitati e il `timestamp` del giorno corrente.
 - *Scadenza Assoluta (Data):* Consente la scrittura manuale in formato italiano (GG/MM/AAAA), la quale viene normalizzata dal codice nella stringa standard ISO AAAA-MM-GG richiesta dal database.

5. Flussi di Navigazione e Architettura dei Canali

Il sistema di navigazione di *MealCraft* non si sviluppa su una transizione lineare e isolata tra schermate, bensì adotta un’architettura radiale e interconnessa facente capo a un unico stato centralizzato, implementata mediante il meccanismo di file-based routing fornito da `Expo Router`.

Il modello poggia su due pilastri infrastrutturali:

- **useAppContext (Il Motore):** Il `Context` centralizzato memorizza in memoria volatile e sincronizza in modo persistente tramite `AsyncStorage` lo stato globale delle quattro macro-aree dati dell’applicazione.
- **router.push con Parametrizzazione URL (I Canali):** Lo scambio di messaggi e direttive tra i diversi nodi della navigazione avviene tramite la serializzazione di coppie chiave-valore all’interno delle stringhe di query degli URL.

L’ecosistema software esegue la sua computazione principalmente attraverso tre macro-flussi operativi interconnessi:

5.1 Il Flusso di Pianificazione Alimentare (Planner → Picker → Planner)

Traccia l’esperienza utente durante l’aggiornamento e la composizione del diario alimentare:

- **Fase 1 (Emissione della Richiesta):** L’utente seleziona uno slot temporale vuoto all’interno di `PlannerScreen`, aprendo la modale inferiore. Sceglie la sorgente dei dati (*Ricette* o *Dispensa*).
- **Fase 2 (Navigazione Vincolata):** Il router esegue il push verso la schermata di destinazione inserendo nell’URL le chiavi `pickerMode: 'true'` e il relativo `mealType`.

- **Fase 3 (Selezione e Serializzazione):** La schermata ricevente (`RecipesScreen` o `PantryScreen`) muta la UI esponendo il controllo `SELEZIONA`. All’attivazione del trigger, l’oggetto selezionato viene convertito in formato stringa JSON e rispedito al mittente tramite navigazione di ritorno.
- **Fase 4 (Iniezione e Sanificazione):** `PlannerScreen`, intercettato il focus di ritorno tramite `useFocusEffect`, esegue il parsing dell’oggetto, vi ancora un `instanceId` univoco per permettere duplicazioni legittime dello stesso alimento nella stessa giornata e aggiorna lo stato globale del piano. Subito dopo, invoca la pulizia dei parametri dell’URL per prevenire loop di inserimento al successivo cambio di focus.

5.2 Il Flusso di Approvvigionamento e Carico (Shopping → Dispensa)

Regola l’ingresso e la transizione di stato delle materie prime acquistate:

- **Fase 1 (Consolidamento Lista):** In `ShoppingScreen` vengono raccolti i prodotti mancanti inseriti manualmente o derivati da algoritmi di calcolo automatici provenienti da altre aree del software.
- **Fase 2 (Rettifica e Computo Temporale):** La spunta dei prodotti avvia la modale di acquisto, dove l’utente valida le quantità reali e definisce le metriche di scadenza (assolute o relative).
- **Fase 3 (Transizione di Stato Globale):** La pressione del comando *“Sposta in Dispensa”* non esegue una navigazione fisica, ma scatena una transizione logica nel `Context`: gli elementi vengono rimossi dal vettore `shoppingList` ed inseriti simultaneamente nel vettore `pantry` tramite la funzione `addToPantry()`. Il salvataggio immediato in `AsyncStorage` propaga istantaneamente i dati aggiornati, rendendoli visibili sia in dispensa sia nel *picker* del planner.

5.3 Il Flusso di Monitoraggio Centrale (Home → Navigazione Radiale)

L’introduzione della `HomeScreen` trasforma la topologia di navigazione da lineare a radiale, posizionandola come un *Hub* centrale di smistamento e analisi:

- **Fase 1 (Query Reattiva e Aggregazione):** All’avvio dell’applicazione, la Home esegue un controllo incrociato istantaneo sullo stato globale. Mappa l’agenda giornaliera, calcola le metriche dei consumi e attiva le notifiche per i prodotti critici o mancanti.
- **Fase 2 (Deep Linking Diretto):** Le schede analitiche fungono da vettori di lancio: cliccando su una criticità, il sistema esegue un `router.push` mirato verso la sezione operativa competente (`/pantry`, `/planner`, `/recipes`), ottimizzando l’esperienza utente.
- **Fase 3 (Ritorno e Allineamento Passivo):** Al termine delle operazioni nei moduli periferici, il ritorno dell’utente alla Home determina un aggiornamento passivo e immediato dell’interfaccia. Senza alcuna necessità di passaggio di parametri nell’URL, i moduli di calcolo interni interrogano lo stato globale mutato, ridisegnando istantaneamente i grafici e rimuovendo gli allarmi precedentemente risolti.

6. Organizzazione dei Dati e Strutture Core

La gestione dei dati all'interno di *MealCraft* è interamente centralizzata all'interno dell'*AppProvider*, il quale opera come un database locale in memoria dell'applicazione, sincronizzato in modo deterministico e persistente sulla memoria flash del dispositivo ospite tramite la libreria *AsyncStorage*.

Al fine di garantire elevate prestazioni e un basso accoppiamento software, le cinque entità principali che compongono l'architettura dei dati locali sono state ingegnerizzate seguendo strutture informative specifiche e mirate al loro contesto d'uso:

1. **Dashboard Analitica (index):** Questa entità non memorizza dati propri in modo persistente sul disco, ma agisce come un motore di calcolo puramente reattivo. Sfruttando gli hook di ciclo di vita, aggrega i dati grezzi provenienti dalle altre sezioni per generare oggetti chiave-valore temporanei (*productCounts*, *ingredientCounts*) e matrici di dati geometrici (*visualSegments*), necessari per il rendering in tempo reale di statistiche, grafici nativi e moduli di allarme.
2. **Ricettario Digitale (recipes):** La struttura è organizzata come un array di oggetti JSON globali, in cui ogni record rappresenta una ricetta autosufficiente e indipendente. *Integrazione Dataset di Base:* Oltre alle ricette create dinamicamente dall'utente, il sistema consuma in modalità di sola lettura le informazioni memorizzate nel file *constants/recipes.ts*. Questo file funge da seed iniziale del database locale e ospita 75 ricette predefinite (suddivise rigorosamente in 15 elementi per categoria), strutturate secondo il medesimo schema JSON utilizzato per i record dinamici (comprendente ingredienti, dosi, grammature e procedimento dettagliato).
3. **Gestione Dispensa (pantry):** L'inventario dei prodotti fisici presenti nella cucina dell'utente è modellato come un array di oggetti, dove ogni elemento mappa un lotto alimentare. Per ottimizzare la consistenza dei dati, il Context implementa una logica di aggregazione algoritmica di tipo *case-insensitive*: prima di effettuare un inserimento, il sistema verifica le stringhe testuali normalizzate per sommare le quantità ed evitare la ridondanza o la duplicazione accidentale di record aventi lo stesso nome.
4. **Lista della Spesa (shopping):** A livello di persistenza nel database locale, l'entità è strutturata come un array di oggetti estremamente snello e indicizzato (*id*, *nome*, *preso*). Tuttavia, a livello di presentazione visiva, la schermata operativa rimodella temporaneamente questo vettore piatto in una struttura complessa a sezioni (un array di oggetti contenente intestazioni e dati ramificati) utile a categorizzare dinamicamente i prodotti in base alla loro specifica area merceologica.
5. **Calendario dei Pasti (plan):** Rappresenta la struttura informativa più complessa e articolata dell'applicazione. È ingegnerizzata come un oggetto chiave-valore basato su una mappa nidificata a tre livelli gerarchici relazionati alle coordinate temporali:

Data (Chiave) → Tipo Pasto (Chiave) → Array di Elementi (Valore)

Per consentire la flessibilità del piano alimentare, ogni singolo record inserito viene marcato da una stringa alfanumerica univoca denominata *instanceId*. Questo accorgimento permette all'utente di aggiungere più volte lo stesso identico alimento o ricetta all'interno dello stesso slot temporale, consentendo al software di identificarli, modificarli o rimuoverli singolarmente senza generare conflitti di puntamento o collisioni di chiavi nella UI.

7. Wireframe e Layout dell'Applicazione

In questa sezione sono presentati i Wireframe statici dell'applicazione, che illustrano l'interfaccia utente delle cinque schermate principali e l'organizzazione logica dei componenti grafici all'interno del layout.

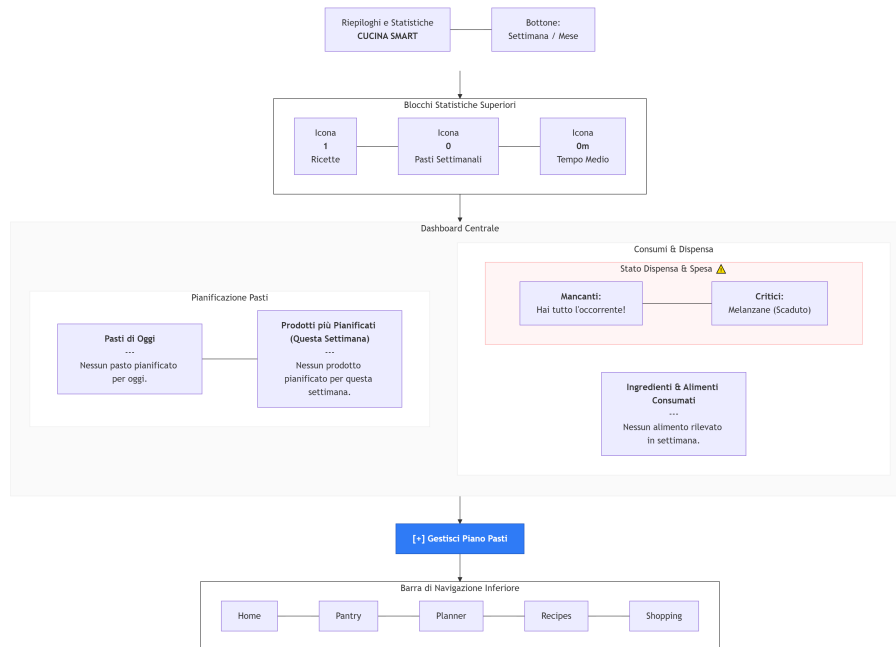


Figura 1: Wireframe statico schermata Index / Home.

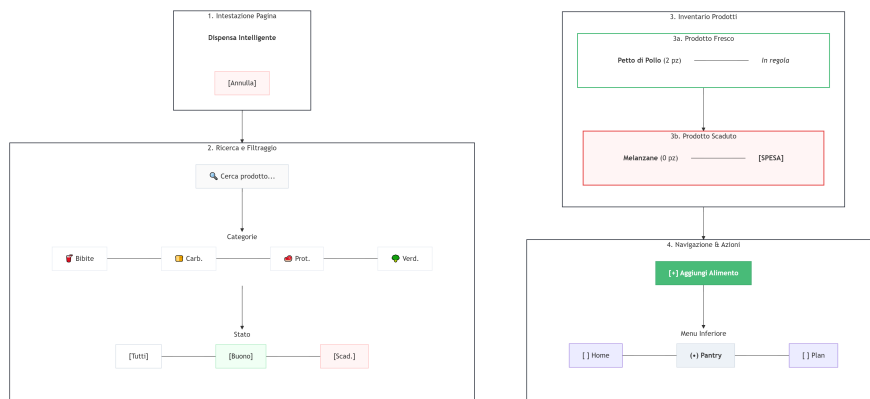


Figura 2: Wireframe statico schermata Pantry / Dispensa.

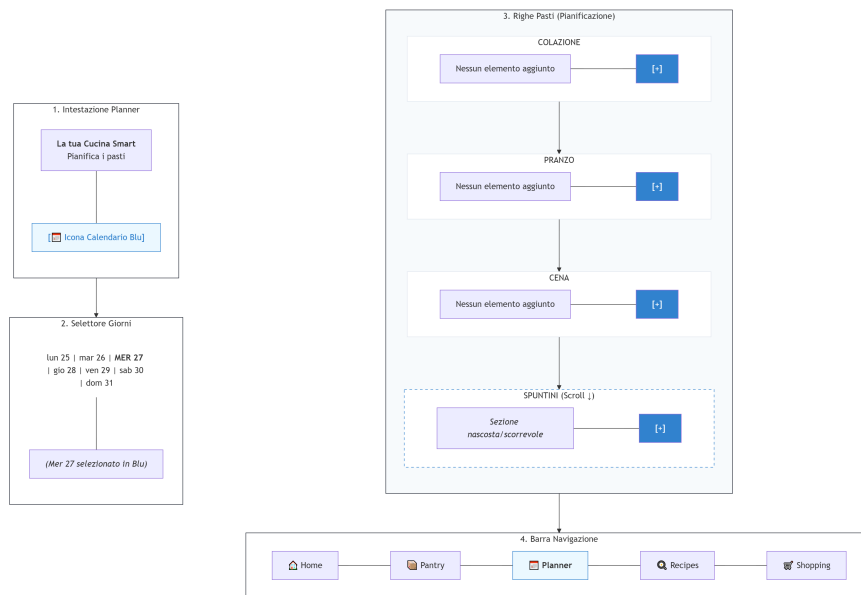


Figura 3: Wireframe statico schermata Planner / Pianificazione.

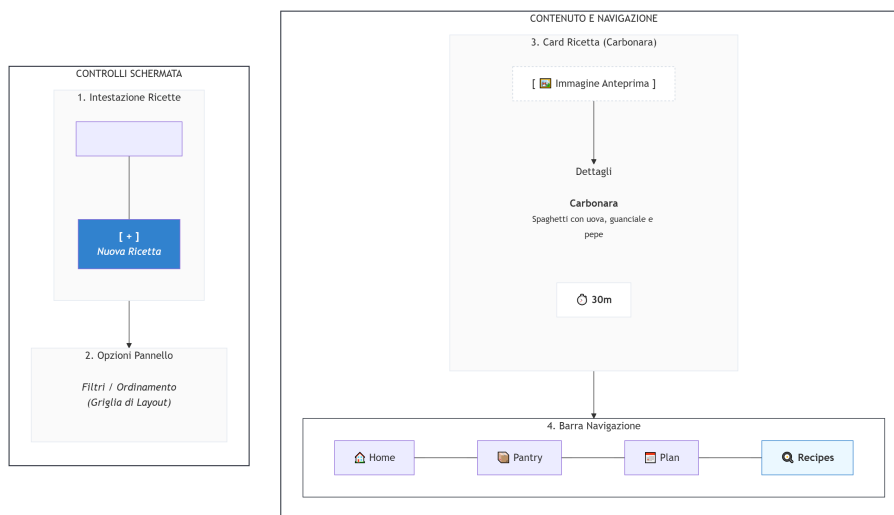


Figura 4: Wireframe statico schermata Recipes / Ricettario.

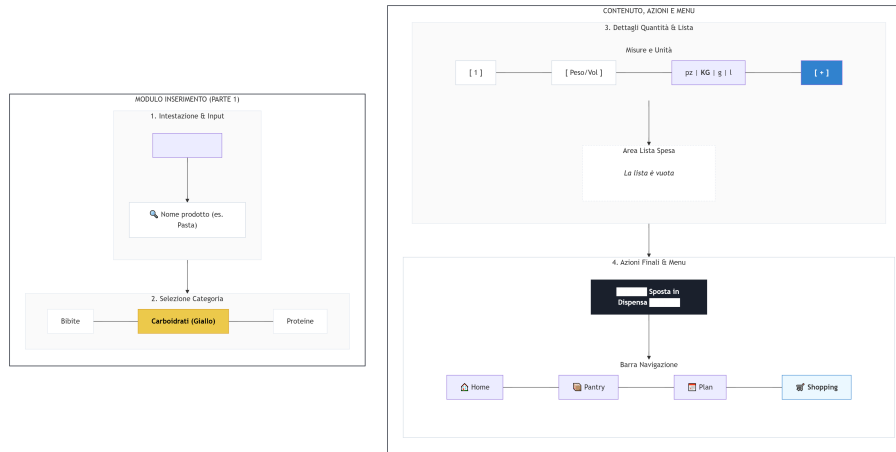


Figura 5: Wireframe statico schermata Shopping / Lista della Spesa.

8. Wireflow e Diagrammi di Flusso

In questa sezione vengono mappati i diagrammi di interazione (Wireflow) dell'applicazione che descrivono in modo analitico lo scambio di dati, gli stati dinamici e il passaggio di parametri reattivi tra le diverse schermate, tracciando l'esperienza d'uso nei tre flussi logici principali.

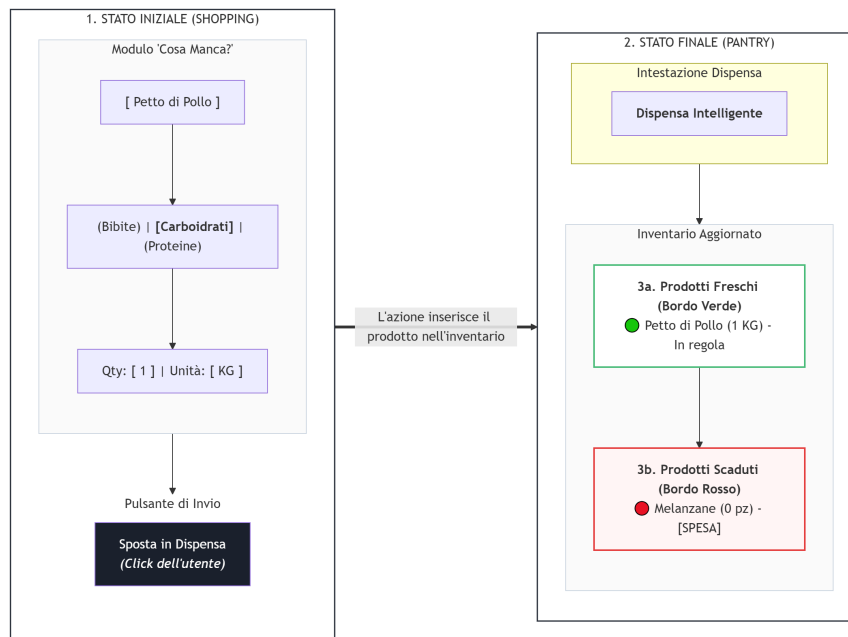


Figura 6: Wireflow di aggiunta pasto da Planner a Pantry / Recipes in Modalità Selezione (Picker Mode).

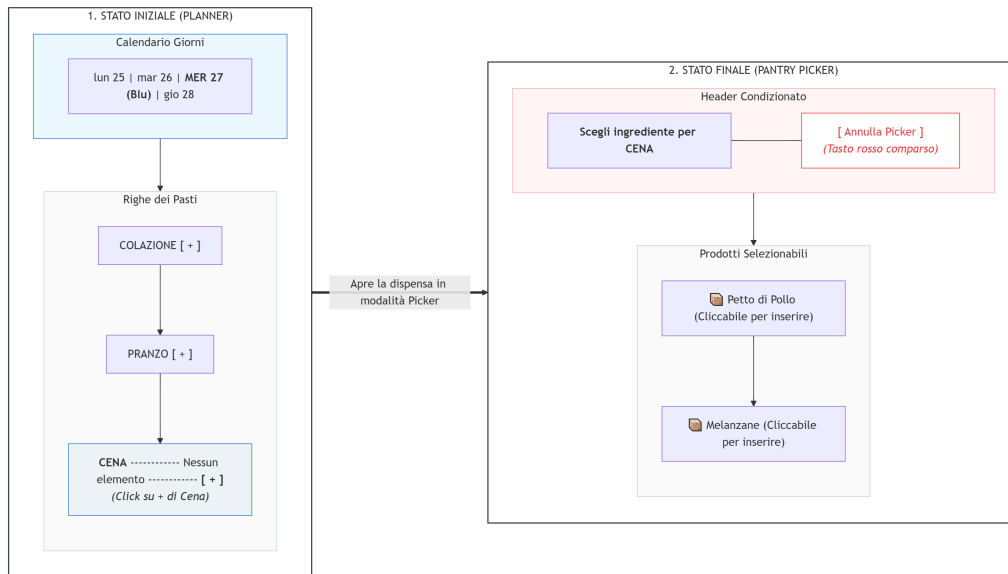


Figura 7: Wireflow di rilevamento criticità e passaggio automatico dal modulo Prodotto Scaduto (Pantry) alla Lista della Spesa (Shopping).

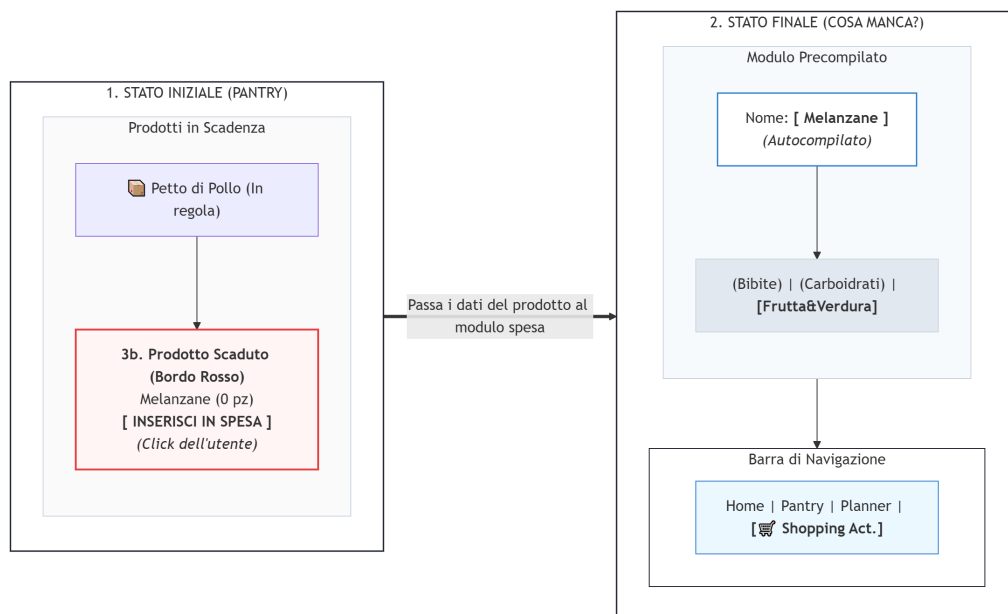


Figura 8: Wireflow di transizione di stato globale e caricamento dei prodotti acquistati dalla Lista della Spesa (Shopping) alla Dispensa (Pantry).

9. Framework Utilizzato e Stack Tecnologico

L'applicazione *MealCraft* è sviluppata interamente all'interno dell'ecosistema **React Native**, sfruttando la suite di strumenti fornita dalla piattaforma **Expo** e implementando il sistema di navigazione nativo basato su file, **Expo Router**.

L'architettura poggia su tre componenti tecnologici fondamentali:

1. **React Native (Il Core):** Framework open-source ingegnerizzato da Meta che consente lo sviluppo cross-platform (iOS e Android) partendo da un unico codice sorgente in JavaScript/TypeScript. A differenza dei framework ibridi basati su *WebView*, React Native converte le istruzioni in componenti grafici reali e nativi del sistema operativo ospite (es. traduzione in *UICollectionView* su iOS e *RecyclerView* su Android tramite astrazioni come *SectionList*), garantendo elevate prestazioni prestazionali e fluidità a 60 FPS.
2. **Expo (L'Ecosistema):** Piattaforma evoluta che astrae la complessità della gestione manuale delle cartelle di build native (*/ios* e */android*), eliminando la necessità di configurare direttamente codice Swift/Objective-C o Java/Kotlin. Expo fornisce un set di API unificate, stabili e ottimizzate (tra cui il pacchetto grafico `@expo/vector-icons` per la gestione dei glifi d'interfaccia), accelerando il ciclo di vita del software.
3. **Expo Router (La Navigazione):** Framework di routing di ultima generazione che introduce nello sviluppo mobile il paradigma del *File-Based Routing* (derivato da Next.js in ambito web). L'alberatura fisica dei file all'interno della directory del progetto definisce matematicamente le rotte dell'applicazione. Consente una gestione del flusso dichiarativa e disaccoppiata basata su URL parametrizzati (es. `/recipes?pickerMode=true`), standardizzando lo scambio di istruzioni inter-schermata.

9.1 Librerie Principali e Dipendenze

Il file di configurazione del progetto (`package.json`) include le seguenti librerie core deputate alla gestione della logica di business e della presentazione visiva:

- **react:** Libreria cardine per la strutturazione dell'interfaccia a componenti reattivi e per la manipolazione degli stati locali e memorizzati (tramite i React Hooks di base `useState`, `useEffect`, `useCallback` e `useMemo`).
- **react-native:** Framework di astrazione per la generazione dei componenti grafici di basso livello integrati nel sistema mobile (es. `View`, `Text`, `TextInput`, `TouchableOpacity`).
- **expo-router:** Modulo software per la gestione dei flussi di navigazione ad albero e del *deep linking* nativo basato su stringhe di query parametriche.
- **@react-native-async-storage/async-storage:** Sistema di memorizzazione locale di tipo chiave-valore, asincrono e persistente, utilizzato come database locale per la serializzazione persistente dei dati sulla memoria non volatile dello smartphone.
- **@expo/vector-icons:** Hub di distribuzione vettoriale per i font di icone, configurato nello sviluppo di *MealCraft* per consumare la collezione di glifi *Ionicons*.

9.2 Motivazione delle Scelte Architetture

Durante la fase di *R&D* (Ricerca e Sviluppo), il gruppo ha analizzato i vincoli di traccia e le necessità prestazionali dell'applicazione, operando scelte architetture mirate a garantire efficienza computazionale e manutenibilità:

- **Stack Cross-Platform (React Native + Expo):** La scelta è stata guidata dalla necessità di ottimizzare i tempi di sviluppo attraverso il riutilizzo del codice (*Single Codebase*). Expo è stato preferito per la stabilità del suo runtime e per la rimozione dei conflitti di compilazione legati ai file di configurazione nativi dei singoli OS.
- **Navigazione Disaccoppiata (Expo Router):** L'adozione del routing basato su URL minimizza la dipendenza accoppiata (*tight coupling*) tra le pagine. Le schermate periferiche (es. *pantry*, *recipes*) non necessitano di referenziare funzioni interne del calendario per restituire un elemento selezionato, ma si limitano a leggere e iniettare parametri nella stringa dell'URL.
- **State Management Centralizzato (React Context API):** Al fine di evitare il fenomeno degradante del *prop drilling* (passaggio continuo di proprietà attraverso componenti intermedi), lo stato è stato isolato in un unico **AppProvider** globale posizionato alla radice del software. Questo assicura che qualsiasi mutazione atomica si propaghi istantaneamente in modo sincrono a tutte le view (Home, Planner, Dispensa, Spesa).
- **Persistenza Automatizzata (AsyncStorage + Promise.all):** Il salvataggio dei vettori di stato è stato centralizzato tramite un osservatore d'effetto (**useEffect**). Per non inficiare le prestazioni di avvio (*App Launch Time*), il ripristino delle chiavi di memoria avviene in parallelo tramite l'invocazione di **Promise.all**, riducendo il tempo totale di caricamento a un singolo ciclo di clock asincrono.
- **Struttura Dati ad Accesso Diretto per il Planner (Mappa Nidificata):** La modellazione del calendario dei pasti tramite un oggetto chiave-valore (indicizzato per stringhe di data) invece di un array piatto rappresenta una scelta strutturale strategica. Questa organizzazione riduce la complessità computazionale di ricerca nella Home e nel Planner da un costo lineare $\mathcal{O}(N)$ a un costo costante $\mathcal{O}(1)$, eliminando la necessità di scansionare l'intero database per trovare i pasti odierni.
- **Architettura dei Dati Derivati e Caching (useMemo):** Per preservare la reattività dell'interfaccia utente nella **HomeScreen**, i dati dei grafici e gli alert predittivi non vengono salvati nel database (operazione che creerebbe ridondanza e disallineamento dei dati), ma computati "al volo". L'utilizzo intensivo dell'hook **useMemo** funge da cache locale, impedendo ricalcoli CPU-intensive a meno che i vettori sorgente nel Context non subiscano variazioni effettive.

9.3 Gestione della Persistenza dei Dati

La persistenza locale è ingegnerizzata all'interno dell'**AppProvider** per operare in modalità totalmente asincrona, trasparente e automatizzata in background, garantendo la totale integrità dei dati alla chiusura forzata dell'applicazione.

Il ciclo di vita della persistenza si articola in tre fasi logiche:

1. **Serializzazione e Storage:** Poiché `AsyncStorage` accetta unicamente stringhe di testo come input, i dati strutturati ed eterogenei (array di oggetti e mappe tridimensionali) vengono serializzati tramite il metodo nativo `JSON.stringify()` prima della transizione di scrittura, e deserializzati tramite `JSON.parse()` in fase di lettura.
2. **Salvataggio Condizionato in Background:** Un effetto di ascolto (`useEffect`) monitora il vettore delle dipendenze globali [`pantry`, `recipes`, `shoppingList`, `plan`]. Al fine di prevenire la corruzione distruttiva del database, l'algoritmo esegue un controllo booleano sulla variabile di controllo `isLoading`: il salvataggio su disco si attiva esclusivamente se `isLoading == true`. Questa precauzione impedisce che lo stato vuoto di pre-inizializzazione del runtime vada a sovrascrivere accidentalmente i dati reali precedentemente memorizzati sul dispositivo durante le sessioni d'uso passate.
3. **Inizializzazione Parallela Multitasking:** All'avvio dell'applicazione, il software avvia il ripristino leggendo contemporaneamente le quattro chiavi di memoria dedicate tramite la gestione di promesse concorrenti:

```
Promise.all([@pantry, @all_recipes, @shopping, @plan])
```

Una volta completata la risoluzione del thread asincrono, gli stati reattivi vengono popolati e il flag `isLoading` viene commutato a `true`, sbloccando la visualizzazione della UI ed evitando la visualizzazione di schermate d'interfaccia orfane di dati.

10. Implementazione e Struttura del Software

10.1 Organizzazione del Codice (Separation of Concerns)

L'architettura del codice sorgente è rigorosamente suddivisa in tre livelli logici indipendenti per garantire scalabilità, isolamento dei bug e modularità:

1. **Livello dei Dati e Logica di Business (/context):** Centralizzato nel file `AppContext.tsx`. Esclude la logica visiva e si occupa esclusivamente del mantenimento degli stati globali reattivi, della gestione della persistenza in background su `AsyncStorage` e dell'esposizione dei metodi atomici di modifica *CRUD* (`addRecipe`, `deleteRecipe`, `addToPantry`, `updatePlan`).
2. **Livello di Presentazione e Routing (/app):** Sfrutta il File-Based Routing. La cartella `app/(tabs)/` implementa il layout della barra di navigazione inferiore (*Bottom Tab Navigation*). Le singole schermate (`index`, `pantry`, `planner`, `recipes`, `shopping`) operano come componenti funzionali puri ed estremamente leggeri (*Smart/Dumb Component Pattern*): non detengono dati di stato logico, ma si limitano a consumare le informazioni distribuite dal contesto mediante l'invocazione dell'hook personalizzato `useAppContext()`.
3. **Moduli di Supporto e Ottimizzazione (/components):** La cartella ospita i componenti visuali riutilizzabili atomici (*Atomic Design Concept*) estratti dalle view principali per massimizzare il riutilizzo del codice. L'ottimizzazione computazionale è demandata a logiche di memorizzazione in cache (`useMemo` e `useCallback`) per azzerare i cicli di *re-render* superflui causati dall'aggiornamento dello stato globale.

10.2 Componenti e Widget Grafici Principali

L'interfaccia utente è stata strutturata combinando primitive grafiche native di React Native e widget ingegnerizzati ad hoc dal gruppo di sviluppo:

- **Componenti Strutturali Nativi:** `SafeAreaView` (per l'ancoraggio dei contenuti all'interno dei confini fisici dello schermo, computando automaticamente la presenza di *notch*, fotocamere integrate o barre di sistema), `ScrollView` (per l'abilitazione dello scorrimento verticale e orizzontale reattivo dei flussi informativi) e `View` (il blocco costruttivo fondamentale per la gestione del layout flessibile tramite motori *Flexbox*).
- **Widget di Input e Feedback Nativi:** `Text` (per il rendering e la formattazione tipografica delle stringhe testuali), `TouchableOpacity` (un wrapper ad alta efficienza per la trasformazione di elementi statici in pulsanti interattivi provvisti di feedback visivo di opacità calibrata al tocco) e `StatusBar` (per il controllo dello stile cromatico dei dettagli di sistema del dispositivo).
- **Widget Avanzati:** `SectionList` (utilizzato per la visualizzazione della lista della spesa, ottimizzando il consumo di memoria RAM tramite il rendering dei soli elementi visibili nello *viewport*, organizzati in sezioni merceologiche eterogenee) e `Icons` (per il caricamento del set di icone vettoriali).
- **Componenti Custom Personalizzati:**
 - *Donut Chart (Grafico a Torta):* Widget analitico geometrico sviluppato sovrapponendo cerchi concentrici nidificati (`View`) e applicando rotazioni vettoriali basate su fogli di stile CSS dinamici, calcolando matematicamente i gradi di ampiezza su base 360°.
 - *Istogramma Nativo:* Grafico a barre verticali statiche in cui l'altezza di ciascuna colonna viene determinata dinamicamente in pixel, calcolando la quota proporzionale in base al volume massimo dei prodotti più pianificati.
 - *Toggle Button Temporale:* Selettore custom a due vie per lo switch di stato istantaneo della dashboard (*Settimana/Mese*).
 - *Carosello delle Scadenze:* Modulo a scorrimento orizzontale vincolato (`ScrollView` con proprietà `horizontal`) per la visualizzazione compatta dei lotti critici in dispensa.

10.3 Delucidazioni Tecniche su Blocchi di Codice Complessi

In questa sezione si analizzano nel dettaglio le cinque soluzioni algoritmiche implementate dal gruppo per risolvere altrettante complessità ingegneristiche emerse durante lo sviluppo delle *advanced feature* dell'applicazione.

1. Algoritmo di Frammentazione Ciclica per il Grafico a Torta (index)

La Complessità: Il motore di rendering dei fogli di stile nativi di React Native (`StyleSheet`) non implementa il supporto ai gradienti conici o ai grafici circolari complessi tipici del browser web (`CSS conic-gradient`). L'utilizzo di bordi colorati ruotati nativamente manifesta gravi distorsioni geometriche e sovrapposizioni errate qualora una singola porzione superi l'angolo retto di 90° (corrispondente al 25% del volume totale dei dati).

La Soluzione: È stato sviluppato un algoritmo di scomposizione ciclica basato su un costrutto `while`. Quando il sistema computa la quota percentuale di consumo di

un ingrediente, se l'angolo risultante eccede i 90°, il codice frammenta dinamicamente la fetta in sotto-segmenti d'ampiezza standard inferiore o uguale a 90° (es. una porzione logica da 130° viene scissa in un segmento hardware da 90° e un secondo segmento contiguo da 40°). Il widget accumula progressivamente la rotazione iniziale di partenza (*accumulatedRotation*), garantendo che ogni sotto-elemento grafico si posizioni millimetricamente dove terminava il frammento precedente, ricomponendo visivamente la continuità del cerchio senza distorsioni di rendering.

2. Architettura dei Doppi ID e Generazione di *instanceId* (*AppContext*)

La Complessità: All'interno del calendario alimentare (*PlannerScreen*), l'utente ha la facoltà logica di pianificare più volte lo stesso identico alimento o snack all'interno della stessa giornata o del medesimo slot (es. una confezione di "Yogurt" a colazione e una seconda confezione di "Yogurt" come spuntino pomeridiano). Entrambi i record fanno riferimento alla stessa entità anagrafica della dispensa, condividendo lo stesso codice identificativo primario (*id*: "yogurt"). Un'operazione standard di filtraggio basata esclusivamente sull'ID determinerebbe una cancellazione distruttiva di massa, eliminando entrambe le istanze dal piano anche a fronte della richiesta di rimozione del singolo spuntino.

La Soluzione: Al momento del caricamento di un elemento nel piano alimentare, il metodo del *Context* intercetta l'oggetto sorgente, esegue una clonazione profonda (*deep copy*) della struttura JSON e vi inserisce una chiave identificativa secondaria generata in tempo reale: l'*instanceId*. Questo codice univoco viene strutturato combinando il *timestamp* Unix espresso in millisecondi (*Date.now()*) con una stringa alfanumerica randomica generata dal motore matematico.

```
Oggetto Sorgente: { id: "yogurt" }
                    ↓
Iniezione nel Planner: { id: "yogurt", instanceId: "1716768000_fj823k"
                        }
```

Le operazioni di manipolazione e rimozione operate dal planner bersagliano esclusivamente la chiave *instanceId*, disaccoppiando le istanze temporali e azzerando i conflitti di collisione delle chiavi energetiche o di indicizzazione nella lista.

3. Ordinamento Prioritario e Sottrazione Retroattiva degli Avanzi (*AppContext*)

La Complessità: In presenza di molteplici lotti dello stesso identico prodotto stoccati in dispensa (es. un lotto aperto e parzializzato contenente 60 g di un prodotto e un secondo lotto integro e sigillato da 500 g), l'applicazione di una rigida logica lineare di tipo *FIFO* (First In, First Out) basata puramente sulle date di scadenza risulterebbe inefficiente. Qualora il lotto sigillato presentasse una scadenza antecedente a quella del lotto aperto, l'algoritmo costringerebbe l'utente ad attingere dalla confezione integra. Questo comportamento provocherebbe uno sdoppiamento incontrollato e anti-economico di confezioni aperte contemporaneamente all'interno della cucina reale.

La Soluzione: La funzione di scarico e decurtamento delle quantità è stata ingegnerizzata introducendo un vincolo di priorità condizionato dallo stato di integrità del lotto. L'algoritmo esegue una scansione preliminare identificando i lotti aventi quantità unitaria (*quantità == 1*) ma con un peso effettivo strettamente inferiore al peso standard nominale impostato dalla casa produttrice (indicatore matematico di un prodotto già parzializzato dall'utente). Questi lotti "aperti" vengono estratti e posizionati in cima alla coda di consumo, indipendentemente dalla loro data di scadenza relativa. L'algoritmo

attinge dai lotti integri solo dopo l'avvenuto esaurimento dell'avanzo. Qualora la decurtazione parzializzasse un lotto originariamente integro, il sistema provvede a rimuovere il record, calcola il peso residuo e genera istantaneamente un nuovo lotto marcato come "aperto" provvisto di un ID univoco rigenerato, automatizzando il riciclo logico e visivo dei prodotti in cucina.

4. Cleanup del Ciclo di Vita del Selettore Inter-schermata (`closePicker`)

La Complessità: Quando l'utente attiva la modalità di selezione (`PickerState`) per associare una ricetta o un ingrediente a uno slot del Planner, il software muta lo stato globale del contesto per configurare la UI delle schermate periferiche. Se l'utente decide di interrompere bruscamente l'operazione cambiando intenzionalmente la scheda di navigazione inferiore (*Bottom Tab*) tramite un tocco involontario o un cambio di contesto, prima di aver premuto esplicitamente i tasti di conferma o annullamento, lo stato di selezione rimarrebbe memorizzato stabilmente in memoria RAM. Al successivo ritorno sul Planner, l'interfaccia risulterebbe inconsistente, presentando un pannello orfano bloccato su dati obsoleti legati a sessioni passate.

La Soluzione: Per preservare la consistenza della *User Experience*, è stata implementata la funzione di smontaggio retroattivo (`cleanup function`) nativa del ciclo di vita dei componenti React, agganciandola alle transizioni di navigazione. All'uscita fisica del focus dalla schermata o alla distruzione del componente visivo del picker, il motore esegue automaticamente la funzione di pulizia: `return () => closePicker();` Il metodo `closePicker()` esegue il reset immediato della variabile di stato `activePicker` impostandola al valore primitivo `null`. Questa sanificazione cancella tempestivamente qualsiasi transazione pendente in background, assicurando che ad ogni nuovo accesso l'interfaccia utente venga inizializzata in uno stato pulito, coerente e totalmente privo di effetti collaterali latenti.

5. Integrazione Avanzata Planner - Shopping List (Smart Sourcing)

Contesto e Obiettivo: All'interno del modulo di pianificazione dei pasti (*Planner*), l'aggiunta di una ricetta non si configura come una mera operazione di inserimento testuale o di calendario, bensì come un'azione che impatta attivamente sulla logica di business e sullo stato delle scorte domestiche. L'obiettivo di questa funzionalità avanzata è **automatizzare il processo di approvvigionamento alimentare**. Quando l'utente inserisce una ricetta all'interno del proprio piano settimanale, il sistema analizza dinamicamente i requisiti della ricetta e le disponibilità reali all'interno della dispensa virtuale. Qualora si riscontrino ingredienti mancanti o insufficienti, l'applicazione provvede ad inserirli in totale autonomia nella lista della spesa (`shoppingList`), sollevando l'utente dall'onere del controllo manuale.

Dettagli Implementativi e Algoritmo di Check: La logica risiede centralizzata all'interno dell'`AppContext` (il motore di stato globale dell'applicazione) all'interno del metodo `updatePlan`. La procedura si articola nelle seguenti fasi sequenziali:

1. **Analisi della Ricetta:** All'atto dell'inserimento di un piatto nel planner, viene intercettato l'array `ingredienti`. Per ciascun elemento viene invocata la funzione di utility `estraiNumeroGrammi(ing.qta)` che, tramite un'espressione regolare (*regex*), converte e normalizza la stringa della quantità in un valore numerico espresso in grammi (o unità base), gestendo in modo robusto caratteri speciali come parentesi e spazi vuoti.
2. **Ispezione Reale della Dispensa:** Il sistema esegue un filtraggio sull'array di stato `pantry`, estraendo tutti i lotti (`PantryItem`) corrispondenti al nome dell'ingrediente

in esame, eseguendo una normalizzazione in *lowercase* e il *trimming* degli spazi vuoti per prevenire errori di battitura o incongruenze di sintassi.

3. **Calcolo delle Rimanenze Totali:** Tramite un'operazione di riduzione (*reduce*), l'algoritmo calcola i grammi totali disponibili in dispensa sommando i vari lotti, pesando opportunamente sia i prodotti a peso specifico (*pesoEffettivo*), sia i prodotti a pezzi interi, secondo la formula:

$$\text{Grammi Totali} = \sum_{\text{lotto} \in \text{lotti}} (\text{Quantità}_{\text{lotto}} \times \text{PesoUnitario}_{\text{lotto}})$$

4. **Risoluzione delle Carenze e Accodamento:** Viene eseguito un confronto logico: se i grammi totali in dispensa sono inferiori ai grammi necessari per la ricetta ($\text{Grammi Totali} < \text{Grammi Necessari}$), l'ingrediente viene inserito in un array temporaneo di carenze denominato *ingredientiMancanti*. Al termine del ciclo, se l'array non è vuoto, viene scatenata la funzione *addToShoppingList(ingredientiMancanti)*.

Smistamento e Gestione dei Prodotti Generati: I prodotti inseriti automaticamente dal Planner confluiscono inizialmente nella schermata *ShoppingScreen* all'interno di una sezione speciale di *fallback* denominata **"Senza Categoria"** (innescata via *expo-router* tramite il passaggio dinamico dei parametri *params.addItem*s).

Per garantire l'integrità strutturale del database della dispensa, l'applicazione impedisce il trasferimento diretto di elementi non categorizzati. L'utente, tramite l'interfaccia di *ShoppingScreen*, è invitato a interagire con la *checkbox* del prodotto generato per aprire un modal di configurazione (*isConfigModalVisible*). In questa sede, l'interfaccia consente di:

- Assegnare l'ingrediente ad una delle macro-categorie merceologiche ufficiali dell'applicazione (*Ortofrutta*, *Latticini*, *Salumi e Formaggi*, *Carne e Pesce*, ecc.).
- Raffinare la quantità effettiva che si intende acquistare, l'unità di misura (*pz*, *kg*, *g*, *l*, *ml*) e il peso specifico.
- Stabilire i criteri di scadenza (espressi in giorni totali o tramite data esplicita).

Solo a seguito di questa validazione e configurazione, il pulsante di chiusura spesa (*finalizeShopping*) convertirà gli elementi acquistati in istanze strutturate *PantryItem* (dotate di ID univoci generati combinando stringhe pseudo-casuali alfanumeriche e *timestamp*), inserendole ufficialmente nei lotti attivi della dispensa in memoria permanente tramite *AsyncStorage*. 6.

Mapping Dinamico dello Stato Globale per la UI del Calendario (*markedDates*)

La Complessità: All'interno del modulo *PlannerScreen*, l'interfaccia deve mostrare visivamente all'utente un indicatore grafico (un "pallino" di notifica) sotto i giorni del calendario che contengono almeno un pasto pianificato. Poiché il componente di calendario nativo richiede una proprietà di configurazione (*markedDates*) strutturata come un dizionario di chiavi rigide in formato stringa ISO AAAA-MM-GG, mappare in tempo reale uno stato globale complesso e profondamente annidato ad ogni minima modifica (inserimento, rimozione o modifica dei pasti) senza causare colli di bottiglia nel thread di rendering della UI rappresenta una criticità di performance.

La Soluzione: Per risolvere il problema riducendo al minimo i ricalcoli superflui, è stata implementata una logica di derivazione dello stato ottimizzata tramite il ciclo di memorizzazione *useMemo*. Il sistema intercetta le variazioni dell'oggetto *pianificazione*

estratto dall'`AppContext`. L'algoritmo esegue un ciclo iterativo `forEach` sulle chiavi temporali; se l'array associato a una determinata data esiste e presenta una lunghezza strettamente maggiore di zero (`length > 0`), viene iniettata dinamicamente una proprietà booleana `marked: true` all'interno del sotto-oggetto della UI. `const markedDates = useMemo(() => { const marked = {}; Object.keys(pianificazione).forEach((data) => { if (pianificazione[data]?.length > 0) marked[data] = { marked: true, dotColor: 'FF6B6B' }; }); return marked; }, [pianificazione]);` Questo oggetto calcolato viene passato direttamente alla proprietà `markedDates` del componente `<Calendar />`, garantendo un aggiornamento visivo immediato e fluido del calendario (e la comparsa del pallino sui giorni attivi) solo quando i dati dei pasti subiscono una reale mutazione di business logic.

11. Considerazioni Finali

Lo sviluppo di MealCraft ha permesso di realizzare un'applicazione mobile solida, reattiva e pienamente rispondente ai requisiti della traccia. L'uso combinato di React Native, Expo Router e Context API ha garantito un'esperienza d'uso fluida sia su Android che su iOS. Il valore principale del lavoro risiede nell'interconnessione dei moduli: la spesa alimenta la dispensa, la dispensa si integra nel planner e la dashboard offre riepiloghi analitici in tempo reale. Le sfide algoritmiche superate — come la frammentazione matematica dei grafici a torta oltre i 90° e il sistema di `instanceId` per evitare conflitti tra pasti duplicati — dimostrano un approccio ingegneristico maturo e consapevole alle problematiche dello sviluppo mobile.

11.1 Possibili implementazioni future

L'architettura pulita e disaccoppiata del codice è già predisposta per accogliere i seguenti upgrade software, emersi in fase di confronto iniziale ma temporaneamente esclusi per circoscrivere l'ambiente iniziale: Evoluzione della Persistenza (Database SQL): Sostituire `AsyncStorage` con un vero database relazionale locale (tramite `expo-sqlite`). Questo permetterà di ottimizzare la memorizzazione, gestire le relazioni native tra tabelle (es. vincolare gli ingredienti delle ricette ai prodotti della dispensa) ed effettuare query di filtraggio e aggregazione molto più rapide. Integrazione Hardware (Scanning Codici QR): Introdurre un modulo di scansione tramite fotocamera (`expo-camera`) in grado di leggere codici QR creati appositamente per identificare i prodotti alimentari. L'utente potrà così inquadrare il codice per inserire o rimuovere istantaneamente un articolo dalla dispensa o dalla lista della spesa, eliminando la necessità di inserimento manuale del testo.